

MÓDULO 6: INTEGRACIÓN GITHUB Y GOOGLE COLAB

Proyecto: Análisis ETL - Entidad Financiera

Prerequisito: MÓDULO 5 completado (base de datos y consultas SQL)

OBJETIVO

Configurar integración entre GitHub y Google Colab para:

- Permitir ejecución del notebook desde cualquier lugar
- Facilitar evaluación del profesor
- Garantizar reproducibilidad del análisis
- Automatizar clonado del repositorio

PASO 1: VERIFICAR REPOSITORIO EN GITHUB

► 1.1 Confirmar que el repositorio está público

En el navegador:

1. Ir a: <https://github.com/Pablo-Tab/ABP-INNOVACION-DATOS>
2. Verificar que NO muestre el candado  (repo privado)
3. Verificar que aparezca el icono  Public

Si el repositorio es privado:

```
# En terminal, navegar al proyecto
cd "C:\Users\PABLO\Desktop\TECNICATURA\ABP INNOVACION"

# Cambiar visibilidad (requiere permisos en GitHub)
# Alternativa: Ir a Settings → Danger Zone → Change visibility → Make public
```

⚠ **IMPORTANTE:** Para que Colab pueda acceder, el repositorio **DEBE** ser público.

► 1.2 Verificar estructura del repositorio

Archivos necesarios en GitHub:

```
ABP-INNOVACION-DATOS/
├── customer_data.csv      ✓ (Datos originales)
├── sales_data.csv         ✓ (Datos originales)
├── datos/
│   └── datos_limpios.csv  ✓ (Datos procesados)
├── notebooks/
│   └── analisis_etl.ipynb ✓ (Notebook principal)
├── sql/
│   ├── schema.sql        ✓ (Esquema BD)
│   ├── consultas.sql     ✓ (Consultas SQL)
│   └── financial_analysis.db ✓ (Base de datos)
├── visualizaciones/      ✓ (Carpeta para gráficos)
├── README.md              ✓ (Descripción del proyecto)
├── requirements.txt       ✓ (Dependencias Python)
└── .gitignore             ✓ (Archivos a ignorar)
```

Verificar en terminal:

```
# Listar archivos en el repositorio
git ls-files

# Verificar que todos los archivos necesarios estén trackeados
```

Salida esperada:

```
customer_data.csv
sales_data.csv
datos/datos_limpios.csv
notebooks/analisis_etl.ipynb
sql/consultas.sql
sql/schema.sql
README.md
requirements.txt
...
```

PASO 2: CONFIGURAR BADGE DE COLAB EN README

► 2.1 Agregar badge al README.md

Abrir **README.md** en VS Code y agregar al inicio:

```
#  Análisis ETL - Entidad Financiera

[![Open In Colab](https://colab.research.google.com/assets/colab-badge.svg)](https://colab.research.google.com/notebooks/analisis_etl.ipynb)

##  Descripción del Proyecto

Proyecto de análisis ETL (Extract, Transform, Load) para una entidad financiera...
```

Componentes del badge:

Elemento	Valor
Imagen	<code>https://colab.research.google.com/assets/colab-badge.svg</code>
URL de Colab	<code>https://colab.research.google.com/github/{usuario}/{repo}/blob/{branch}/{ruta}</code>
Usuario GitHub	<code>Pablo-Tab</code>
Repositorio	<code>ABP-INNOVACION-DATOS</code>
Branch	<code>main</code>
Ruta del notebook	<code>notebooks/analisis_etl.ipynb</code>

Formato del URL de Colab:

```
https://colab.research.google.com/github/Pablo-Tab/ABP-INNOVACION-DATOS/blob/main/notebooks/a
```

► 2.2 Actualizar README.md completo

Archivo: `README.md`

```
#  Análisis ETL - Entidad Financiera
```

```
[![Open In Colab](https://colab.research.google.com/assets/colab-badge.svg)](https://colab.research.google.com/)
```

```
##  Descripción del Proyecto
```

Proyecto de análisis ETL (Extract, Transform, Load) realizado para el curso **Tratamiento y Seguridad de la Información**

El proyecto analiza datos de ventas de una entidad financiera, respondiendo a los **Items 4a-4e**

- Extracción y limpieza de datos con Pandas
- Análisis exploratorio de datos (EDA)
- Visualizaciones con Matplotlib
- Implementación de base de datos SQLite
- Consultas SQL analíticas

```
---
```

```
##  Ejecución Rápida
```

```
### Opción 1: Google Colab (Recomendado)
```

1. Hacer clic en el badge **"Open in Colab"** arriba
2. Ejecutar las celdas en orden (Runtime → Run all)
3. El notebook clonará automáticamente el repositorio

```
### Opción 2: Ejecución Local
```

```
```bash
```

```
Clonar repositorio
```

```
git clone https://github.com/Pablo-Tab/ABP-INNOVACION-DATOS.git
```

```
cd ABP-INNOVACION-DATOS
```

```
Crear entorno virtual
```

```
python -m venv venv
```

```
venv\Scripts\activate # Windows
```

```
source venv/bin/activate # Linux/Mac
```

```
Instalar dependencias
```

```
pip install -r requirements.txt
```

```
Abrir Jupyter
```

```
jupyter notebook notebooks/analisis_etl.ipynb
```



## Estructura del Proyecto

```

ABP-INNOVACION-DATOS/
├── customer_data.csv # Datos de clientes (99,457 registros)
├── sales_data.csv # Datos de ventas (99,457 registros)
├── datos/
│ └── datos_limpios.csv # Dataset procesado (99,338 registros)
├── notebooks/
│ └── analisis_etl.ipynb # Notebook principal con análisis completo
├── sql/
│ ├── schema.sql # Esquema de base de datos
│ ├── consultas.sql # 17 consultas analíticas SQL
│ └── financial_analysis.db # Base de datos SQLite
├── visualizaciones/ # Gráficos generados (13 imágenes PNG)
│ ├── 4a_metodos_pago.png
│ ├── 4b_genero_compras.png
│ ├── 4c_facturacion_categoria.png
│ └── ...
├── DOCUMENTACION_TECNICA/ # Guías paso a paso
│ ├── 01_CONFIGURACION_ENTORNO.md
│ ├── 02_CARGA_DATOS.md
│ ├── 03_LIMPIEZA_TRANSFORMACION.md
│ ├── 04_ANALISIS_VISUALIZACIONES.md
│ └── 05_IMPLEMENTACION_SQL.md
├── README.md # Este archivo
├── requirements.txt # Dependencias Python
└── .gitignore # Archivos a ignorar en Git

```



## Resultados Principales

---

► **Item 4a: Método de Pago Más Utilizado**

**Respuesta:** Cash (44.72% de transacciones)

► **Item 4b: Género con Más Compras**

**Respuesta:** Female (59.77% de transacciones)

► **Item 4c: Categoría con Mayor Facturación**

**Respuesta:** Clothing (\$113.8M, 33.60% del total)

► **Item 4d: Ticket Promedio por Ubicación**

**Respuesta:** Ankara (\$3,459.23 promedio)

► **Item 4e: Categoría con Mayor Precio Promedio**

**Respuesta:** Technology (\$3,157.35 promedio)

---

## Tecnologías Utilizadas

Tecnología	Versión	Uso
Python	3.11+	Lenguaje principal
Pandas	2.1.1	Manipulación de datos
NumPy	1.26.0	Cálculos numéricos
Matplotlib	3.8.0	Visualizaciones
SQLite3	3.x	Base de datos
Jupyter	Latest	Entorno interactivo
Git/GitHub	Latest	Control de versiones

## Estadísticas del Dataset

- **Registros totales:** 99,338 transacciones
- **Periodo:** Enero 2021 - Marzo 2023
- **Clientes únicos:** 4,913
- **Facturación total:** \$338,707,188.45
- **Ticket promedio:** \$3,409.67
- **Calidad de datos:** 99.88% completos



## Autor

### Pablo Tab

- GitHub: [@Pablo-Tab](#)
- Proyecto: Tecnicatura Superior en Ciencia de Datos e IA
- Curso: TSCDIA 2025



## Licencia

Este proyecto es de uso académico para la Tecnicatura Superior en Ciencia de Datos e Inteligencia Artificial.



## Agradecimientos

- Profesores del curso TSCDIA
- Compañeros de la Tecnicatura
- Comunidad de Python y Jupyter

**Última actualización:** 8 de noviembre de 2025

```

```

```
2.3 Guardar y subir README.md
```

```
```powershell
```

```
# Guardar cambios
```

```
git add README.md
```

```
git commit -m "Actualizar README con badge de Colab y documentación completa"
```

```
git push origin main
```

PASO 3: CONFIGURAR CELDA DE COLAB EN EL NOTEBOOK

► 3.1 Agregar celda de detección de Colab

En el notebook `analisis_etl.ipynb`, crear PRIMERA celda (después del título):

Celda Markdown:

```
# ⚙ Configuración para Google Colab

**IMPORTANTE:** Si ejecutas este notebook en Google Colab, ejecuta la siguiente celda para clonar el repositorio de GitHub.

Si ejecutas localmente en Jupyter, **OMITE** la siguiente celda.
```

Celda Python (código):

```
# =====
# CONFIGURACIÓN AUTOMÁTICA PARA GOOGLE COLAB
# =====

import sys
import os

# Detectar si estamos en Google Colab
IN_COLAB = 'google.colab' in sys.modules

if IN_COLAB:
    print('=' * 80)
    print('🌐 EJECUTANDO EN GOOGLE COLAB')
    print('=' * 80)

    # Clonar repositorio si no existe
    repo_name = 'ABP-INNOVACION-DATOS'

    if not os.path.exists(repo_name):
        print(f'\n📄 Clonando repositorio {repo_name}...')
        !git clone https://github.com/Pablo-Tab/ABP-INNOVACION-DATOS.git
        print('✅ Repositorio clonado exitosamente')
    else:
        print(f'\n✅ Repositorio {repo_name} ya existe')

    # Cambiar directorio de trabajo
    os.chdir(repo_name)
    print(f'\n📁 Directorio de trabajo: {os.getcwd()}')

    # Verificar archivos críticos
    critical_files = [
        'customer_data.csv',
        'sales_data.csv',
        'notebooks/analisis_etl.ipynb'
    ]

    print('\n🔍 Verificando archivos críticos:')
    all_ok = True
    for file in critical_files:
        exists = os.path.exists(file)
        status = '✅' if exists else '❌'
        print(f'    {status} {file}')
        if not exists:
            all_ok = False
```

```

if all_ok:
    print('\n✅ ENTORNO CONFIGURADO CORRECTAMENTE')
    print('Puedes continuar ejecutando el resto del notebook')
else:
    print('\n⚠️ ADVERTENCIA: Algunos archivos no se encontraron')
    print('Verifica la estructura del repositorio')

print('=' * 80)

else:
    print('💻 Ejecutando en entorno local (Jupyter)')
    print('✅ No se requiere configuración adicional')
    
```

¿Qué hace esta celda?

Línea	Función
'google.colab' in sys.modules	Detecta si está en Colab
!git clone https://...	Clona el repositorio
os.chdir(repo_name)	Cambia al directorio del repo
os.path.exists(file)	Verifica que archivos existan

Salida en Colab:

```
=====
🌐 EJECUTANDO EN GOOGLE COLAB
=====
```

```
📁 Clonando repositorio ABP-INNOVACION-DATOS...
Cloning into 'ABP-INNOVACION-DATOS'...
remote: Enumerating objects: 145, done.
remote: Counting objects: 100% (145/145), done.
remote: Compressing objects: 100% (98/98), done.
remote: Total 145 (delta 47), reused 145 (delta 47), pack-reused 0
Receiving objects: 100% (145/145), 9.23 MiB | 5.12 MiB/s, done.
Resolving deltas: 100% (47/47), done.
☑ Repositorio clonado exitosamente
```

```
📁 Directorio de trabajo: /content/ABP-INNOVACION-DATOS
```

```
🔍 Verificando archivos críticos:
☑ customer_data.csv
☑ sales_data.csv
☑ notebooks/analisis_etl.ipynb
```

```
☑ ENTORNO CONFIGURADO CORRECTAMENTE
Puedes continuar ejecutando el resto del notebook
=====
```

Salida en local:

```
💻 Ejecutando en entorno local (Jupyter)
☑ No se requiere configuración adicional
```

► 3.2 Modificar celda de carga de CSVs

Actualizar la celda que carga los archivos CSV:

```
# =====
# CARGA DE DATOS
# =====

import pandas as pd
import os

print('=' * 80)
print('CARGANDO DATASETS')
print('=' * 80)

# Definir rutas posibles (Colab vs Local)
possible_paths = [
    ('customer_data.csv', 'sales_data.csv'),          # Colab (ya en raíz)
    ('../customer_data.csv', '../sales_data.csv'),    # Local (desde notebooks/)
    ('data/customer_data.csv', 'data/sales_data.csv') # Alternativa
]

# Buscar archivos
customers_file = None
sales_file = None

for customer_path, sales_path in possible_paths:
    if os.path.exists(customer_path) and os.path.exists(sales_path):
        customers_file = customer_path
        sales_file = sales_path
        break

# Validar que se encontraron los archivos
if customers_file is None or sales_file is None:
    print('\n❌ ERROR: No se encontraron los archivos CSV')
    print('\nRutas buscadas:')
    for cp, sp in possible_paths:
        print(f'    - {cp} | {sp}')
    print('\n⚠️ Asegúrate de ejecutar la celda de configuración de Colab')
    raise FileNotFoundError('Archivos CSV no encontrados')

# Cargar archivos
print(f'\n📁 Cargando archivos:')
print(f'    - Clientes: {customers_file}')
print(f'    - Ventas: {sales_file}')

df_customers = pd.read_csv(customers_file)
df_sales = pd.read_csv(sales_file)
```

```
print(f'\n✅ Datasets cargados exitosamente')
print(f' 📊 Clientes: {len(df_customers):,} registros, {df_customers.shape[1]} columnas')
print(f' 📊 Ventas: {len(df_sales):,} registros, {df_sales.shape[1]} columnas')
print('=' * 80)
```

Ventajas de este código:

- ☒ Detecta automáticamente la ubicación de los archivos
- ☒ Funciona en Colab y local sin cambios
- ☒ Muestra error claro si no encuentra archivos
- ☒ Informa qué ruta se está usando



PASO 4: PROBAR EN GOOGLE COLAB

► 4.1 Abrir notebook en Colab

Opción A: Desde GitHub

1. Ir a: <https://github.com/Pablo-Tab/ABP-INNOVACION-DATOS>
2. Click en [notebooks/analisis_etl.ipynb](#)
3. Esperar que GitHub muestre el preview
4. Click en "Open in Colab" (ícono arriba a la derecha)

Opción B: URL directo

```
https://colab.research.google.com/github/Pablo-Tab/ABP-INNOVACION-DATOS/blob/main/notebooks/analisis\_etl.ipynb
```

Opción C: Desde el badge del README

1. Ir a: <https://github.com/Pablo-Tab/ABP-INNOVACION-DATOS>
2. Click en el badge naranja "Open in Colab"

► 4.2 Ejecutar notebook en Colab

Pasos:

1. Conectar a runtime:

- Click en "Connect" (arriba a la derecha)
- Esperar que aparezca RAM y Disk disponibles

2. Ejecutar celda de configuración:

- Click en la primera celda de código (configuración Colab)
- Click en  o presionar **Shift + Enter**
- Verificar que clone el repositorio correctamente

3. Ejecutar todo el notebook:

- Menú: **Runtime** → **Run all**
- O presionar: **Ctrl + F9** (Windows) / **Cmd + F9** (Mac)

4. Monitorear ejecución:

- Cada celda mostrará  cuando termine
- Revisar que no haya errores 

5. Verificar outputs:

- Items 4a-4e deben mostrar respuestas
 - Gráficos deben generarse correctamente
 - Consultas SQL deben ejecutarse sin errores
-

► 4.3 Diferencias Colab vs Local

Aspecto	Google Colab	Jupyter Local
Instalación	No requiere	Requiere Python + Jupyter
Hardware	GPU/TPU gratis	Recursos locales
Persistencia	Archivos se pierden al cerrar	Archivos permanentes
Internet	Requerido	Opcional
Colaboración	Fácil de compartir	Requiere Git
Directorio	<code>/content/</code>	Directorio del proyecto
Paquetes	Pre-instalados	Manual con pip



PASO 5: COMPARTIR EL NOTEBOOK

► 5.1 Link directo de Colab

URL para compartir:

<https://colab.research.google.com/github/Pablo-Tab/ABP-INNOVACION-DATOS/blob/main/notebooks/>

Enviar al profesor:

Estimado profesor,

El notebook está disponible en Google Colab:

https://colab.research.google.com/github/Pablo-Tab/ABP-INNOVACION-DATOS/blob/main/notebooks/analisis_etl.ipynb

Para ejecutarlo:

1. Click en el link
2. Click en "Connect" (arriba a la derecha)
3. Menú: Runtime → Run all

El notebook clonará automáticamente el repositorio y ejecutará todo el análisis.

Repositorio completo: <https://github.com/Pablo-Tab/ABP-INNOVACION-DATOS>

Saludos,
Pablo Tab

► 5.2 Agregar link al README

Actualizar sección en README.md:

```
## 🔗 Links Importantes
```

- **📄 Notebook en Colab:** [Abrir en Google Colab](https://colab.research.google.com/github/Pablo-Tab/ABP-INNOVACION-DATOS/blob/main/notebooks/analisis_etl.ipynb)
- **📁 Repositorio GitHub:** [ABP-INNOVACION-DATOS](https://github.com/Pablo-Tab/ABP-INNOVACION-DATOS)
- **📄 Notebook archivo:** [analisis_etl.ipynb](notebooks/analisis_etl.ipynb)



PASO 6: GUARDAR TODO Y HACER COMMIT

FINAL

```
# Guardar todos los cambios
git add .
git commit -m "Configurar integración completa con Google Colab - notebook listo para evaluación"
git push origin main
```



PASO 7: VERIFICACIÓN FINAL

► Checklist de verificación:

- ☒ Repositorio es público en GitHub
- ☒ Badge de Colab está en README.md
- ☒ README.md tiene documentación completa
- ☒ Celda de configuración Colab agregada al notebook
- ☒ Celda de carga CSV detecta rutas automáticamente
- ☒ Notebook probado exitosamente en Colab
- ☒ Todos los archivos necesarios están en GitHub
- ☒ Link directo de Colab funciona correctamente
- ☒ Commit final realizado y pusheado



TROUBLESHOOTING

► Problema 1: “Repository not found” en Colab

Causa: Repositorio privado o URL incorrecta

Solución:

1. Verificar que el repo sea público
2. Verificar URL: `https://github.com/Pablo-Tab/ABP-INNOVACION-DATOS`
3. Verificar branch: `main` (no `master`)

► Problema 2: “FileNotFoundError” al cargar CSVs

Causa: Rutas incorrectas o archivos no en GitHub

Solución:

```
# Verificar contenido del directorio
!ls -la

# Verificar directorio actual
import os
print(os.getcwd())

# Listar archivos CSV
!find . -name "*.csv"
```

► Problema 3: "ModuleNotFoundError" para algún paquete

Causa: Paquete no instalado en Colab

Solución:

```
# Instalar paquete faltante
!pip install nombre_paquete

# Ejemplo:
!pip install pandas==2.1.1
```

► Problema 4: Notebook muy lento en Colab

Causa: Recursos limitados del runtime gratuito

Soluciones:

1. Cambiar tipo de runtime:

- Runtime → Change runtime type
- Hardware accelerator: GPU o TPU

1. Reducir tamaño de datos:

- Usar `.sample()` para análisis rápido

```
python df_sample = df_merged.sample(n=10000, random_state=42)
```

2. Optimizar código:

- Evitar loops innecesarios
- Usar operaciones vectorizadas de Pandas



COMANDOS GIT PARA COLAB

Comando	Función
<code>!git clone URL</code>	Clonar repositorio
<code>!git status</code>	Ver estado del repo
<code>!git pull origin main</code>	Actualizar con cambios remotos
<code>!git log --oneline -5</code>	Ver últimos 5 commits
<code>!ls -la</code>	Listar archivos (Linux)
<code>!pwd</code>	Mostrar directorio actual



CONCEPTOS APRENDIDOS

Concepto	Descripción
Colab Runtime	Entorno de ejecución temporal en la nube
Badge de Colab	Botón para abrir notebook directamente
Clonado automático	Copiar repo desde GitHub a Colab
Detección de entorno	Diferenciar Colab vs local
Rutas relativas	Acceder a archivos sin ruta absoluta
Repositorio público	Accesible sin autenticación



PRÓXIMOS PASOS

Ver **MÓDULO 7: SOLUCIÓN DE PROBLEMAS** para:

- Errores comunes y soluciones
- Optimización de código
- Mejores prácticas
- Consejos para presentación

Documento creado: 8 de noviembre de 2025

Parte de: Documentación Técnica Completa - Proyecto ETL TSCDIA